Formal Verification of the Brevis Pico RISC-V zkVM

Technical Report



NETHERMIND

10th of June, 2026

1 Executive Summary

This report summarises the engagement performed by Nethermind on formally verifying, in the Lean proof assistant, the Brevis Pico RISC-V zkVM implementations of both the RV32IM and RV64IM variants of the RISC-V instruction set against the Lean RISC-V specification.

The work done in this engagement was based on the official Brevis Pico repository (commit `e128062`, the head of PR #101, dating from June 5th, 2026), and resulted in the following deliverables:

1. a mechanism for extracting Plonky3 constraints from Pico chips;
2. a comprehensive verification infrastructure written in the Lean proof assistant (version 4.28.0-rc1), available at `github.com/NethermindEth/pico-fv`; and
3. as part of this infrastructure, Lean proofs that the Pico zk implementations of *all* RV32IM and RV64IM opcodes satisfy their Lean RISC-V specifications. Concretely:
   - for **RV32IM**, the 45 opcodes in the base ISA, comprising 27 ALU (ADD, ADDI, SUB, XOR, XORI, OR, ORI, AND, ANDI, SLT, SLTI, SLTU, SLTIU, SLL, SLLI, SRL, SRLI, SRA, SRAI, MUL, MULH, MULHU, MULHSU, DIV, DIVU, REM, REMU), 10 control-flow (AUIPC, LUI, BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL, JALR), and 8 memory opcodes (LW, LH, LHU, LB, LBU, SW, SH, SB);
   - for **RV64IM**, the 62 opcodes in the RV64IM ISA: the 45 RV32IM opcodes above (with LW now signed-extended per the RV64 spec), the 5 word-variant ALU opcodes (ADDW, SUBW, SLLW, SRLW, SRAW), the 4 word-variant immediate opcodes (ADDIW, SLLIW, SRLIW, SRAIW), the 5 word-variant MUL/DIV opcodes (MULW, DIVW, DIVUW, REMW, REMUW), and the 3 new memory opcodes (LWU, LD, SD).

   The ECALL opcode is the only RV32IM/RV64IM opcode left out of scope, as it is not required for L1 proving.

For each verified opcode, the equivalence theorem states that, under the chip's row constraints, bus assumptions, and program well-formedness, executing one Sail step of the corresponding instruction produces the same post-state as applying the chip's emitted CPU-bus operations to a Sail state. As a concrete example, the RV64IM equivalence theorem for ADD (opcode 0), the full source of which is available here, is:

```
theorem equivalence_ADD
  [Field ExtF]
  (air : Valid_CpuChip FKB ExtF)
  (row : ℕ)
  (h_row : row ≤ air.last_row)
  (h_is_real : air.cols.is_real row 0 = 1)
  (h_all_axioms : ∀ row ≤ air.last_row, bus_row_axioms (bus_row air row))
  (h_all_ext : ∀ row ≤ air.last_row, bus_row_ext_asm (bus_row air row))
  (h_all_wf : ∀ row ≤ air.last_row, bus_row_wf_asm (bus_row air row))
  (h_all_wf_asrt : ∀ row ≤ air.last_row, bus_row_wf_asrt (bus_row air row))
  (h_all_constraints : ∀ row ≤ air.last_row, CpuChip.Extraction.allHold air row)
  (h_opcode : air.cols.instruction.opcode row 0 = 0 ∧ air.cols.opcode_selector.imm_c row 0 = 0)
  (h_bus_hypotheses : (operation_list_effect (CPU_bus_operations air row) state).hypotheses)
  (risc_v_assumptions : RISC_V_assumptions state mstatus pmaRegion misa mseccfg)
:
  let rd  : regidx := .Regidx (air.cols.instruction.op_a_0 row 0).val
  let rs1 : regidx := .Regidx (air.cols.instruction.op_b_0 row 0).val
  let rs2 : regidx := .Regidx (air.cols.instruction.op_c_0 row 0).val
  execute_instruction (instruction.RTYPE (rs2, rs1, rd, rop.ADD)) state =
    (operation_list_effect (CPU_bus_operations air row) state).state
```

Running the proofs. Assuming that an installation of Lean's lake package manager is present on the system, the infrastructure and all of the proofs can be built using:

```
lake update
lake exe cache get!
lake build
```

in the root folder of the repository, where the first two lines are superfluous but allow for a faster initial build.

1.1 Verification context, assumptions, and caveats

The verification was performed in the context of future participation of Pico in Ethereum L1 proving, for which the Ethereum Foundation (EF) is developing a proposal for the standardisation of the RISC-V target architecture for Ethereum with the goal of defining a minimal RISC-V instruction set for zkEVMs that would like to prove Ethereum blocks. This proposal, at the time of writing this report, makes the following recommendations that are relevant for this engagement:

EF1: that the supported set of instructions should be RV64IM: Pico supports both RV32IM and RV64IM variants, and **both are verified in full as part of this engagement**;

EF2: that the only supported privileged mode should be machine mode (M);

EF3: that no interaction with the environment using system calls should be required;

EF4: that the memory should be treated as flat, without advanced use of the memory management unit (MMU) and related concepts, such as paging; and

EF5: that misaligned memory accesses should be allowed: Pico currently supports only aligned memory accesses.

On the other hand, the Lean RISC-V specification models almost all of the mechanisms, both mandatory and optional, described by the RISC-V standard and various extensions. In line with EF recommendations and the design choices made by Pico developers, we modify the 32-bit and 64-bit variants of the Lean RISC-V specification as follows:

S1: We disable the Zca RISC-V extension by setting its support to `false` here (32-bit) and here (64-bit). The Zca extension enables the RISC-V C instruction set, which allows for higher data compression. This extension is not implemented by Pico and is not required for L1 proving.

S2: We disable the Zicfilp RISC-V extension by setting its support to `false` here (32-bit) and here (64-bit). The Zicfilp extension enforces forward-edge control-flow integrity by introducing a landing-pad instruction, LPAD, which must be placed at the program locations that are valid targets of indirect jumps or calls. Pico does not support the LPAD instruction, and there is no indication that this support is required for L1 proving. In the proofs, this is relevant only for the JALR opcode.

S3: We disable the RISC-V core-local interruptor (CLINT), which is a memory-mapped peripheral that handles software and timer interrupts, by setting its size to zero here (32-bit) and here (64-bit). This mechanism is not required for L1 proving, and disabling it is also in line with recommendation EF4 and the uniform way in which Pico models memory. In the proofs, this is relevant for all of the memory-manipulating opcodes.

S4: We disable the RISC-V Physical Memory Protection (PMP) mechanism here (32-bit) and here (64-bit). The PMP is not modelled by Pico, and disabling it is in line with recommendation EF4. In the proofs, this is relevant for all of the memory-manipulating opcodes.

Further, we make the following assumptions on the RISC-V state in our Lean development, formalised here (32-bit) and here (64-bit). The two definitions are identical except for the address-space upper bound (`Pico_address_space_size_32 = 2130706440` for the 32-bit variant, matching the KoalaBear prime plus three; `Pico_address_space_size_64 = 2^48` for the 64-bit variant, matching Pico's three-limb 48-bit address representation). We reproduce the 64-bit version verbatim below; the assumptions are:

A1: We 1) set the privileged mode to be machine mode; and 2) do not allow for this privilege to be overridden by setting the `MPRV` bit of the `mstatus` register to zero. This is in line with EF2.

A2: We set up the Physical Memory Attributes (PMA) in line with EF2. Specifically:

1. We declare there to be a single PMA region. This is to make sure that all correctly aligned memory accesses will be executed, disallowing cases in which the load or store would cross beyond the end of a PMA region (for example, if the size of a PMA region is not a multiple of four, and a four-byte load was requested from the address that is the multiple of four closest to its end).
2. We set the base address of this single PMA region to be zero, and its size to be at least `Pico_address_space_size`: 2130706440 (`KoalaBearPrime` $+ 3$) bytes for the 32-bit variant, 2^{48} bytes for the 64-bit variant. In doing so, we ensure that the actual address space of Pico is supported (for more details, see finding 1 of RV32IM below).
3. We set this entire single PMA region to be readable, writable, and with misaligned accesses allowed, in line with the EF zkVM standard recommendations. We note, however, that Pico at the time of writing only supported aligned memory accesses.

A3: We disable the host/target mechanism of RISC-V. This is in line with EF3, as support for this mechanism should not be required for L1 proving.

A4: In order for all opcodes to be executable by the Lean RISC-V specification, we assume the existence of: 1) the `misa` register, required for most control-flow opcodes; and 2) the `mseccfg` register, required for JALR.

```
-- Pico address space size. Pico's RV64 `Addr<T> = [T; 3]` representation
-- caps addresses at 48 bits, so we use `2^48` as the tight upper bound.
notation "Pico_address_space_size_64" ⇒ 2 ^ 48

def RISC_V_assumptions
  (state : PreSail.SequentialState RegisterType Sail.trivialChoiceSource)
  (mstatus : RegisterType Register.mstatus)
  (pmaRegion : PMA_Region)
  (misa : RegisterType Register.misa)
  (mseccfg : RegisterType Register.mseccfg)
: Prop :=
  -- Assumption A1.1: machine privilege
  Sail.readReg Register.cur_privilege state = EStateM.Result.ok Privilege.Machine state ∧
  -- Assumption A1.2: MPRV bit of the mstatus register not set
  (Sail.readReg Register.mstatus state = EStateM.Result.ok mstatus state ∧
   BitVec.extractLsb 17 17 mstatus = 0#1) ∧
  -- A2.1 : Single PMA region
  Sail.readReg Register.pma_regions state = EStateM.Result.ok [ pmaRegion ] state ∧
  -- A2.2 : with base 0 and at least of the required size
  pmaRegion.base = 0 ∧
  Pico_address_space_size_64 ≤ pmaRegion.size.toNat ∧
  -- A2.3 : with all addresses readable and writable, and misaligned accesses allowed
  pmaRegion.attributes.readable ∧
  pmaRegion.attributes.writable ∧
  pmaRegion.attributes.misaligned_fault = misaligned_fault.NoFault ∧
  -- Assumption A3: no host-target interface
  Sail.readReg Register.htif_tohost_base state = EStateM.Result.ok .none state ∧
  -- Assumption A4.1: misa register exists
  Sail.readReg Register.misa state = EStateM.Result.ok misa state ∧
  -- Assumption A4.2: mseccfg register exists
  Sail.readReg Register.mseccfg state = EStateM.Result.ok mseccfg state
```

Finally, when it comes to the underlying infrastructure, we assume the following:

I1: correctness of the theory and implementation of the Pico lookup argument and bus interaction mechanism;

I2: correctness of the theory and implementation of the proof system utilised by Pico;

I3: correctness of the RISC-V Lean specification with respect to the official RISC-V standard; and

I4: correctness of Lean's kernel.

1.2 Summary of findings

In the course of this engagement, we uncovered **13 chip-side soundness gaps** in the RV64IM Pico implementation, distributed across the `LtChip` (1), `ShiftRightChip` (3), `SLLChip` (1), `JalrChip` (1), `DivRemChip` (4), and `MemoryReadWriteChip` (3). **All 13 have been closed upstream** by Pico PR #101 "[don't-merge] fix soundness and constraints for formal verification". Each gap is catalogued individually below, together with the chip, a one-line description of the failure mode, and the upstream fix commit (where available). For the `MemoryReadWriteChip` gaps, a developer-facing write-up was additionally produced and is available in the repository as `RV64_MemoryReadWriteChip_soundness_gaps.md`.

RV64IM gaps closed upstream by Pico PR #101.

LT-1 (`LtChip`). The output limb `a[0]` of the chip was not constrained to equal `lt_signed.result.bit`, so the signed-comparison output was effectively unwitnessed. Affects SLT (opcode 8).

SR-1 (`ShiftRightChip`). For the W-variants SRLW/SRAW, the upper output limbs `a[0]`/`a[1]` were left as free witnesses (the chip even contained a corresponding `TODO` comment). Fix: derive these limbs from `limb_result`

and the MSB-carrying limb `higher_limb[1]`, avoiding the sign-extended-upper-limbs witness.

SR-2 (`ShiftRightChip`). The per-limb decomposition into `lower_limb[i]` / `higher_limb[i]` was byte-range-checked but never tied back to the input bytes `b[i]`. Fix: 4 new constraints `b[i] · v_0123 = higher_limb[i] · 65536 + lower_limb[i] · v_0123`.

SR-3 (`ShiftRightChip`). The scalar shift amount `c` (computed from `c_bits`/`v_0123`) and the value `c_for_lookup` used for the CPU multiset lookup were never tied by an `assert_eq`. Fix: the single constraint `assert_eq(c, c_for_lookup[0])`.

SLL-1 (`SLLChip`). The auxiliary witness `shift_u16` was neither boolean-checked nor tied to `c_bits[4..6]` for SLL/SLLW, allowing an arbitrary effective shift amount. Fix: boolean check on `shift_u16` plus an explicit binding to `c_bits[4..6]`.

JALR-1 (`JalrChip`). Bit 0 of the computed target address was not cleared, violating the RISC-V specification of JALR (which masks the low bit of the computed target: `(rs1 + imm) & ~1`). Without this mask the prover could witness a target address with `bit = 0`.

DR-1 (`DivRemChip`). The `LtWordU16Gadget::eval` call site never asserted `bit = 1`, so the gadget only enforced the bi-implication `bit ↔ (a < b)` and the load-bearing $|r| < |c|$ check was effectively absent. This allowed multiple (q, r) pairs to satisfy the carry chain. Fix: `builder.when(rem_check_mult).assert_one(bit)`.

DR-2 (`DivRemChip`). `quot_msb` was never passed to `U16MSBGadget::eval`, leaving it as a free witness. The prover could therefore forge the sign extension of the upper limbs of the quotient in word-variant operations.

DR-3 (`DivRemChip`). The upper limbs `c[2]`, `c[3]` of the divisor were unconstrained when `is_word_operation = 1`. The prover could pick `c[2]` $\neq$ `0` while `c[0] = c[1] = 0`, making the `is_c_0` flag spuriously false and bypassing the divide-by-zero special case.

DR-4 (`DivRemChip`). The upper limbs `b[2]`, `b[3]` of the dividend were unconstrained when `is_word_operation = 1`. The prover could pick a large quotient such that $c \cdot q$ overflowed into `b[2]`/`b[3]`, satisfying $c \cdot q + r = b$ with an incorrect quotient.

MRW-1 (`MemoryReadWriteChip`). No constraint tied the (now-removed) column `addr_aligned` to the chip's `addr_word`, so the State bus could emit arbitrary aligned addresses. The RV32IM analogue, `constraint_12`, ties `addr_aligned` to `addr_word` explicitly; the RV64IM chip had dropped it. Fix (commit `db682d9c`): delete the `addr_aligned` column entirely and compute the aligned address inline as `[addr_word[0] - offset_sum, addr_word[1], addr_word[2]]`.

MRW-2 (`MemoryReadWriteChip`). The constraints binding the `op_a` payload all carried a `(1 - op_a_0)` factor, so when `op_a_0 = 1` (i.e. the load target was `x0`) the four limbs `value_0..3` of the memory value were left unbounded. This blocked the load equivalence proofs. Fix (commit `7860af12`): four extra constraints `op_a_0 · op_a_val[i] = 0` for $i \in \{0, 1, 2, 3\}$.

MRW-3 (`MemoryReadWriteChip`). For LB/LBU, only the low byte of the selected limb was range-checked via a byte-bus entry; the high byte `(selected_limb − selected_limb_low_byte)` · 2122383361 was left unbounded (the byte-bus entry carried $x_3 = 0$ literally). This blocked one sub-case of `spec_LBU` as well as both `bus_equiv_LB` and `bus_equiv_LBU`. Fix (commit `fc197c0b`): replace the single-byte `slice_range_check_u8` with `looking_rangecheck(U8Range, ..., low_byte, high_byte, ...)`, so that both bytes are now range-checked.

Original RV32IM findings. The three findings below were the first to be uncovered, prior to PR #101, in the course of formalising RV32IM. They all relate either to a general property of the program counter (finding 1) or to the `MemoryReadWriteAir` (findings 2 through 4). In the RV64IM `MemoryReadWriteChip`, the analogues of findings 2 and 4 (i.e. gaps MRW-1 and MRW-2 above) have been closed upstream by PR #101; the analogue of finding 3 (the explicit constraint that the memory address is ≥ 32, in order to disambiguate the register-vs-memory regions of the `State` bus) **remains open**, both in RV32IM and in RV64IM, and is flagged in-source via the `FIXME` attached to the explicit `ASSUMPTION : ¬ (air.cols.addr_aligned row 0 < 32)` parameter to the `bus_equiv` lemma in the RV32IM `MemoryReadWriteAir` file.

The other airs/chips have been implemented fully correctly, following the RISC-V specification.

1. The program counter must be enforced during the transpilation phase to have an upper bound that does not allow overflow (e.g., 2^{30}).

2. Currently, what is declared to be the maximal memory address is set to be $\leq$ `BabyBearPrime`. This setting, however, allows one to have an aligned memory address of `BabyBearPrime` $- 1$, which means that the effective largest memory address accessible to a read or write is `BabyBearPrime` $+ 2$. This can be corrected in two ways:
   (a) the maximal memory address could be constrained to be strictly less than a specified limit that must be divisible by four; or
   (b) the documentation should be adapted to correctly reflect the actual maximal memory address.
3. Given the design of the `State` bus, which captures both register writes (< 32) and memory writes (≥ 32), in order to ensure that the meaning of the `Memory` bus is aligned with the intended meaning of the `State` bus, the memory address read from or written to must explicitly be constrained to be greater than or equal to 32 in the constraints. Otherwise, one could be attempting to write into memory but instead be writing to a register. This also means that memory at addresses from 0-31 is fully inaccessible.

2 Verification methodology

In this section we walk through, end to end, how the Pico chips are made to talk to the Sail RISC-V specification, how the Pico buses are formalised, and how the per-opcode correctness theorems are stated and proved. The verification infrastructure is structurally identical for RV32IM and RV64IM: the same generic bus-entry interface, the same operation-effect model on the RISC-V state, and the same per-opcode workflow. The two developments differ only in three mechanical ways:

1. the machine word is `BitVec 32` in RV32IM and `BitVec 64` in RV64IM;
2. the per-bus payloads grow to accommodate the wider PC/address space and the extra RV64-only opcodes (ADDW, SUBW, ADDIW, SLLW, SRLW, SRAW, SLLIW, SRLIW, SRAIW, MULW, DIVW, DIVUW, REMW, REMUW, LWU, LD, SD); and
3. operand limbs widen to u16 (so `Word<T>` becomes a four-limb u16 vector rather than a four-limb u8 vector).

For the rest of this section we therefore describe the RV64IM development only, and link each snippet directly to the corresponding source location in the verification infrastructure. The RV32IM counterpart can be found under `PicoFV/RV32IM/` and follows the same layout file-for-file.

2.1 Interface with the RISC-V specification

The starting point is a small algebra of seven elementary operations that together describe every effect any RV64IM opcode in scope can have on the Sail RISC-V state. These operations and their supporting types are defined here:

```
-- Supported widths of memory accesses.
inductive MemoryWidth where
| Byte | Half | Word | Double

def width_of_mem_width (mw : MemoryWidth) :=
  match mw with
  | .Byte   ⇒ 8
  | .Half   ⇒ 16
  | .Word   ⇒ 32
  | .Double ⇒ 64

-- Atomic effects on the Sail RISC-V state used by every RV64IM opcode in scope.
inductive Operation where
| ReadPC     (val : BitVec 64) : Operation         -- read the program counter
| WritePC    (val : BitVec 64) : Operation         -- write the next program counter
| ReadReg    (reg : regidx) (val : BitVec 64) : Operation
| WriteReg   (reg : regidx) (val : BitVec 64) : Operation
-- Read/write of a value of `width_of_mem_width mw` bits at byte address `addr`.
| ReadMem    (addr : ℕ) (mw : MemoryWidth) (val : BitVec (width_of_mem_width mw)) : Operation
| ReadMemAny (addr : ℕ) (mw : MemoryWidth) : Operation
| WriteMem   (addr : ℕ) (mw : MemoryWidth) (val : BitVec (width_of_mem_width mw)) : Operation
```

Two points are worth highlighting. First, memory accesses come in four widths (`Byte` of 8 bits, `Half` of 16, `Word` of 32 and `Double` of 64), corresponding directly to the LB/LBU/SB, LH/LHU/SH, LW/LWU/SW and LD/SD families respectively. The `Double` width is RV64-only. Second, the `ReadMemAny` variant captures load operations whose destination is `x0`: in that case the read value is unconstrained on the bus side (Pico does not bind it) but the load must still "succeed", that is, the bytes must actually be present in the memory map. We track this with the existential "`isSome`"-shape hypothesis below (see also the LB example later in this subsection).

The effect of one operation. Each operation, when applied to a current "effect snapshot" of the Sail state, either learns a new fact about it (reads) or transforms it (writes). We package both into a single record, defined here:

```
structure OperationEffect where
  hypotheses : Prop
  state      : EStateM.Result (Sail.Error exception)
                              (PreSail.SequentialState RegisterType Sail.trivialChoiceSource)
                              ExecutionResult
```

where hypotheses accumulates the facts learnt from reads, and state is the post-state after applying writes. We then define the effect of *one* operation on such a snapshot, defined here (only the structurally interesting cases are shown; the .Half, .Word and .Double memory cases are analogous to the four-byte fragment shown for .Word):

```
def operation_effect
  (result : OperationEffect)
  (entry  : Operation)
: OperationEffect :=
  match result.state with
  -- A previous operation raised an error: propagate it unchanged.
  | .error _ _ ⇒ result
  -- Previous operation succeeded; apply the new one.
  | .ok _ state ⇒
    match entry with
    -- Reading the PC learns its current value; the state is unchanged.
    | .ReadPC val ⇒
      { result with hypotheses := result.hypotheses ∧
                                  Sail.readReg Register.PC state = EStateM.Result.ok val state }
    -- Writing the PC has no precondition; the new state has `nextPC := val`.
    | .WritePC val ⇒
      { result with state := EStateM.Result.map (fun _ ↦ ExecutionResult.Retire_Success ())
                                                (Sail.writeReg Register.nextPC val state) }
    -- Reading a register learns its current value.
    | .ReadReg reg val ⇒
      { result with hypotheses := result.hypotheses ∧
                                  read_xreg (regidx_to_fin reg) state = EStateM.Result.ok val state }
    -- Writing a register transforms the state via `write_xreg`.
    | .WriteReg reg val ⇒
      { result with state := EStateM.Result.map (fun _ ↦ ExecutionResult.Retire_Success ())
                                                (write_xreg (regidx_to_fin reg) val state) }
    -- Reading memory: width-dispatched. For a `Word`-sized read we learn the
    -- four constituent bytes are present at the relevant addresses.
    | .ReadMem addr mw val ⇒
      match mw with
      | .Byte ⇒
        { result with hypotheses := result.hypotheses ∧ state.mem[addr]? = .some val }
      | .Word ⇒
        let data0 := BitVec.extractLsb 7  0  val
        let data1 := BitVec.extractLsb 15 8  val
        let data2 := BitVec.extractLsb 23 16 val
        let data3 := BitVec.extractLsb 31 24 val
        { result with hypotheses := result.hypotheses ∧
                                    state.mem[addr]?     = .some data0 ∧
                                    state.mem[addr + 1]? = .some data1 ∧
                                    state.mem[addr + 2]? = .some data2 ∧
                                    state.mem[addr + 3]? = .some data3 }
      | _ ⇒ ... -- .Half and .Double analogous.
    -- A `ReadMemAny` (loads with `rd = x0`) only commits to bytes being present, not to their value.
    | .ReadMemAny addr mw ⇒
      match mw with
      | .Byte ⇒ { result with hypotheses := result.hypotheses ∧ state.mem[addr]?.isSome }
      | _ ⇒ ... -- the other widths conjoin one `isSome` per byte.
    -- Writing memory inserts the value's bytes, width-dispatched.
    | .WriteMem addr mw val ⇒ ... -- analogous to `ReadMem`, but on the state.
```

The effect of a list of operations. The cumulative effect of a sequence of operations on an initial Sail state is then a left fold of operation_effect over the list, starting from the no-op snapshot, defined here:

```
def operation_list_effect
  (ops   : List Operation)
  (state : PreSail.SequentialState RegisterType Sail.trivialChoiceSource) : OperationEffect :=
  List.foldl operation_effect
    { hypotheses := True, state := EStateM.Result.ok (ExecutionResult.Retire_Success ()) state } ops
```

Per-opcode specifications. For each RV64IM opcode we then write down, explicitly and by hand, the sequence of `Operations` that the opcode is meant to perform on the Sail state. Each per-opcode spec lives in its own file under `PicoFV/RV64IM/RVSail/Opcodes/`. We show five representative examples below: an R-type (ADD), a branch (BNE), an indirect jump (JALR), a load with the `rd = x0` edge case (LB), and an RV64-only store (SD).

The 64-bit ADD, defined here:

```
def spec_ADD (r1 r2 rd : regidx) (pc_val r1_val r2_val : BitVec 64)
: List Operation :=
  [ .ReadPC pc_val,                  -- read the current PC
    .ReadReg r1 r1_val,              -- read register r1, obtaining r1_val
    .ReadReg r2 r2_val,              -- read register r2, obtaining r2_val
    .WritePC (pc_val + 4#64),        -- next PC = current PC + 4
    .WriteReg rd (r1_val + r2_val) ] -- write the sum to register rd
```

BNE, defined here:

```
def spec_BNE (r1 r2 : regidx) (imm : BitVec 13) (pc_val r1_val r2_val : BitVec 64)
: List Operation :=
  -- PC increment: 4 if equal (fall-through), else the sign-extended 13-bit offset.
  let increment := if r1_val == r2_val then 4#64 else BitVec.signExtend 64 imm
  [ .ReadPC pc_val,
    .ReadReg r1 r1_val,
    .ReadReg r2 r2_val,
    .WritePC (pc_val + increment) ]
```

JALR, defined here, note how the low bit of the computed target is masked, in line with the RISC-V specification:

```
def spec_JALR (rs1 rd : regidx) (imm : BitVec 12) (pc_val rs1_val : BitVec 64)
: List Operation :=
  let mask : BitVec 64 := 0xFFFFFFFFFFFFFFFE
  [ .ReadPC pc_val,
    .ReadReg rs1 rs1_val,
    .WritePC (mask &&& (rs1_val + BitVec.signExtend 64 imm)),   -- target with bit 0 cleared
    .WriteReg rd (pc_val + 4#64) ]                              -- return address into rd
```

LB, defined here, illustrating the `rd = x0` edge case (the bus does not constrain the read value when the destination register is `x0`, so the spec only commits to the byte being present in the memory map):

```
def spec_LB (r1 rd : regidx) (imm : BitVec 12) (pc_val r1_val : BitVec 64) (data0 : BitVec 8)
: List Operation :=
  if rd == regidx.Regidx 0#5
  then  -- rd = x0: the byte must exist but its value is irrelevant.
    [ .ReadPC pc_val,
      .ReadReg r1 r1_val,
      .ReadMemAny (r1_val + BitVec.signExtend 64 imm).toNat .Byte,
      .WritePC (pc_val + 4#64) ]
  else  -- rd ≠ x0: read the byte, sign-extend to 64 bits, write to rd.
    [ .ReadPC pc_val,
      .ReadReg r1 r1_val,
      .ReadMem (r1_val + BitVec.signExtend 64 imm).toNat .Byte data0,
      .WritePC (pc_val + 4#64),
      .WriteReg rd (BitVec.signExtend 64 data0) ]
```

SD (RV64-only 8-byte store), defined here:

```
def spec_SD (r1 r2 : regidx) (imm : BitVec 12) (pc_val r1_val r2_val : BitVec 64)
: List Operation :=
  [ .ReadPC pc_val,
    .ReadReg r1 r1_val,
    .ReadReg r2 r2_val,
    .WritePC (pc_val + 4#64),
    .WriteMem (r1_val + (BitVec.signExtend 64 imm)).toNat .Double r2_val ]
```

The Sail-side equivalence lemmas. For each opcode we then prove that applying the operation list against the Sail state yields exactly the same result as executing the corresponding Sail instruction. For ADD, the lemma is stated and proved here:

```
lemma execute_ADD_equiv
  (h : (operation_list_effect (spec_ADD r1 r2 rd pc_val r1_val r2_val) state).hypotheses)
:
  execute_instruction (instruction.RTYPE (r2, r1, rd, rop.ADD)) state =
    (operation_list_effect (spec_ADD r1 r2 rd pc_val r1_val r2_val) state).state
```

where `execute_instruction` is the natural "one step" driver that increments `nextPC` and dispatches to the Sail `execute` function, defined here:

```
noncomputable def execute_instruction
  (instr : instruction) (state : PreSail.SequentialState RegisterType Sail.trivialChoiceSource)
:=
  (do
    Sail.writeReg Register.nextPC (Sail.BitVec.addInt (← Sail.readReg Register.PC) 4)
    LeanRV64D.Functions.execute instr
  ) state
```

These per-opcode equivalence lemmas effectively reduce the global correctness problem to a single, mechanical question for each chip: *does the activity of the chip's bus, viewed as an operation list, agree with the opcode's spec_∗ list?* The rest of this section formalises "the activity of the chip's bus" precisely enough that we can ask, and answer, that question.

2.2 General treatment of buses

Pico uses LogUp-style bus arguments for inter-chip communication and for lookup tables. Each bus carries entries of the form $\langle \text{multiplicity}, \text{data} \rangle$, where multiplicity is typically 0 (no-op), 1 (an air *writes* the entry), or -1 (an air *reads* the entry); the data is a fixed-width vector of field elements whose meaning is bus-specific.

We capture every bus uniformly via a single typeclass, defined in `PicoFV/Fundamentals/Interaction.lean` (shared between RV32IM and RV64IM):

```
class BusEntry (α : Type*) where
  multiplicity : α → F

  data_length  : ℕ
  data         : α → Vector F data_length

  -- Infrastructure-external constraints on bus entries (axiomatic facts that the
  -- bus-emitting Pico infrastructure --- rather than the chips themselves --- enforces).
  axioms       : α → Prop

  -- Chip-external, infrastructure-internal constraints (constraints that the calling
  -- air must verify on read, and may assume on write).
  externals    : α → Prop

  -- Well-formedness properties (assumed on read, asserted on write).
  wf_properties : α → Prop

  -- Conditions under which the externals/well-formedness must be assumed or asserted.
  wf_assume_cond   : α → Prop := fun e ⇒ multiplicity e = -1
  wf_assert_cond   : α → Prop := fun e ⇒ multiplicity e =  1
  ext_assume_cond  : α → Prop := fun e ⇒ multiplicity e =  1
  ext_assert_cond  : α → Prop := fun e ⇒ multiplicity e = -1

  ext_assume (a : α) := ext_assume_cond a → externals a
  ext_assert (a : α) := ext_assert_cond a → externals a
  wf_assume  (a : α) := wf_assume_cond  a → wf_properties a
  wf_assert  (a : α) := wf_assert_cond  a → wf_properties a

  -- Serialisation / deserialisation (with inverses).
```

```
  serialise       (x : α) := (multiplicity x, data x)
  serialiseToList (x : α) := (multiplicity x, (data x).toList)
  deserialise     : F × Vector F data_length → α

  inv_ser_deser (x : F × Vector F data_length) : serialise (deserialise x) = x
  inv_deser_ser {x : α} : deserialise (serialise x) = x
```

The class fixes, for every bus, six things: its data length, its multiplicity projection, three predicates capturing the semantics of the bus (`axioms`, `externals`, `wf_properties`), and a serialise/deserialise pair with proofs that the two are mutual inverses. The defaults attached to `wf_*_cond` and `ext_*_cond` capture the canonical LogUp polarities:

- `externals` are *assumed* when an air writes the entry (multiplicity $= 1$), and *asserted* when an air reads it (multiplicity $= -1$);
- `wf_properties` are *assumed* on read and *asserted* on write, i.e. the writer guarantees well-formedness, the reader gets to depend on it.

This separation of bus semantics into three predicates pays off when we later prove per-opcode equivalence: each chip only needs to discharge a small, local obligation ("my writes are well-formed and satisfy externals"), and may use the corresponding companion hypothesis for entries it reads.

2.3 The Pico buses

Pico uses five buses in the implementation of the RV64IM opcodes relevant for this engagement: `Program`, `ALU`, `Byte`, `State` and `Memory`. All five live in `PicoFV/RV64IM/RVSail/Buses.lean`. We describe each in turn; in every case the semantics of a bus is given by the combination of its axioms, externals, well-formedness predicate, and its contribution to the operation list applied to the Sail state (the latter is deferred to §2.4).

2.3.1 The `Program` bus and the RISC-V instruction transpiler

The `Program` bus is what plugs the Pico runtime into the RISC-V Sail semantics: every instruction to be executed is transpiled into one `Program` bus entry which is then consumed by the CPU chip on the corresponding row. The entry structure, defined here, carries a 43-element payload, consisting of 3 u16 limbs of program counter, the opcode in duplicate, three operands with four u16 limbs each, the `a_is_0` indicator, the two immediate markers `imm_b` / `imm_c`, and 23 opcode selectors:

```
structure ProgramBusEntry where
  multiplicity    : F -- Entry multiplicity
  pc_0 pc_1 pc_2  : F -- Program counter (3 u16 limbs; the RV64 PC address space is 48 bits wide).
  opcode opcode'  : F -- Opcode (emitted twice on the bus: as `opcode` and as a copy `opcode'`).
  a_0 a_1 a_2 a_3 : F -- Operands a/b/c, each a four-limb u16 vector.
  b_0 b_1 b_2 b_3 : F
  c_0 c_1 c_2 c_3 : F
  a_is_0          : F -- Indicator: is the destination register `x0`?
  imm_b imm_c     : F -- Immediate-encoding markers.
  -- Opcode selectors (one-hot, except for the `is_alu` super-selector).
  is_alu is_ecall : F
  is_lb is_lbu is_lh is_lhu is_lw is_lwu is_ld : F   -- `is_lwu`, `is_ld` are RV64-only.
  is_sb is_sh is_sw is_sd : F                        -- `is_sd` is RV64-only.
  is_beq is_bne is_blt is_bge is_bltu is_bgeu : F
  is_jalr is_jal : F
  is_auipc is_unimpl : F
```

The transpiler that produces these entries from Sail instructions is formalised in `Transpiler.transpile_op`, in full faithfulness to its Rust implementation in the Pico codebase. Its signature is:

```
def transpile_op
  (inst         : instruction)        -- RISC-V instruction (Sail-side)
  (multiplicity : FKB)                -- intended program-bus entry multiplicity
  (pc           : Vector FKB 3)       -- current PC, as three u16 limbs
  -- Returns the deserialised program-bus payload together with the precondition
  -- that the Pico infrastructure guarantees about the transpiled entry.
: Option ((FKB × Vector FKB 43) × Prop)
```

The function returns none only for instructions out of scope. One subtlety worth mentioning is that LUI is not given a dedicated chip opcode: since its immediate already arrives correctly shifted in the RISC-V instruction format, the transpiler emits an ADD (opcode 0) with the shifted immediate threaded through `imm_b` / `imm_c`. The transpiler is structured as a single `match` on the Sail instruction constructor; for example, the six branch opcodes BEQ/BNE/BLT/BGE/BLTU/BGEU are handled in one branch, defined here:

```
| .BTYPE (imm, rs2, rs1, op) =>
  let opcode := opcode_of_bop op
  let a_is_0 : FKB := 0
  let imm_KB := imm_to_KB imm true
  .some (
    ( multiplicity,
      #v[
        pc[0], pc[1], pc[2], opcode, opcode,
        reg_to_KB rs1, 0, 0, 0,
        reg_to_KB rs2, 0, 0, 0,
        imm_KB[0], imm_KB[1], imm_KB[2], imm_KB[3],
        a_is_0,
        0, 1,                              -- imm_b = 0, imm_c = 1
        0, 0,                              -- is_alu = 0, is_ecall = 0
        0, 0, 0, 0, 0, 0, 0,               -- load selectors
        0, 0, 0, 0,                        -- store selectors
        (if op == .BEQ  then 1 else 0),
        (if op == .BNE  then 1 else 0),
        (if op == .BLT  then 1 else 0),
        (if op == .BGE  then 1 else 0),
        (if op == .BLTU then 1 else 0),
        (if op == .BGEU then 1 else 0),
        0, 0,                              -- is_jalr, is_jal
        0, 0                               -- is_auipc, is_unimpl
      ]
    ),
    is_register (reg_to_KB rs1) ∧
    is_register (reg_to_KB rs2) ∧
    imm % 2 = 0
  )
```

```
def opcode_of_bop (op : bop) : FKB :=
  match op with
  | .BEQ  => 18 | .BNE  => 19
  | .BLT  => 20 | .BGE  => 21
  | .BLTU => 22 | .BGEU => 23
```

```
def is_register (reg : FKB) : Prop :=
  reg < 32
```

The transpilation thus comes packaged with two precondition facts that the transpiler guarantees: that the source registers `rs1` and `rs2` are valid (i.e. less than 32), and that the immediate is 2-byte aligned (the RISC-V format itself ensures branch immediates are even).

The `Program` bus instance is then defined here:

```
instance ProgramBusEntryInstance
: BusEntry FKB (ProgramBusEntry FKB) :=
{
  multiplicity := fun entry => entry.1,

  data_length := 43,
  data := fun ⟨_, pc_0, pc_1, pc_2, opcode, opcode', a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2,
               c_3, a_is_0, imm_b, imm_c, is_alu, is_ecall, is_lb, is_lbu, is_lh, is_lhu, is_lw, is_lwu,
               is_ld, is_sb, is_sh, is_sw, is_sd, is_beq, is_bne, is_blt, is_bge, is_bltu, is_bgeu, is_jalr,
               is_jal, is_auipc, is_unimpl⟩ =>
          #v[pc_0, pc_1, pc_2, opcode, opcode', a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2,
               c_3, a_is_0, imm_b, imm_c, is_alu, is_ecall, is_lb, is_lbu, is_lh, is_lhu, is_lw, is_lwu,
               is_ld, is_sb, is_sh, is_sw, is_sd, is_beq, is_bne, is_blt, is_bge, is_bltu, is_bgeu, is_jalr,
               is_jal, is_auipc, is_unimpl]

  -- Axioms: when active, the PC is 4-byte aligned. The full 48-bit upper bound follows externally
  -- from each pc-limb being u16-bounded (enforced via the byte-bus `U16Range` lookup).
  axioms := fun entry => (¬ entry.multiplicity = 0) → entry.pc_0.val % 4 = 0

  -- No chip-external constraints.
  externals := fun _ => True
```

```
  -- Well-formedness is inherited from the transpiler: a `ProgramBusEntry` is well-formed iff it can
  -- be produced by `transpile_op` on some Sail instruction.
  wf_properties := ProgramBusEntry.operand_properties

  ...
}
```

where operand_properties is the existential "some Sail instruction transpiles to this row" predicate, defined here:

```
def ProgramBusEntry.operand_properties (entry : ProgramBusEntry FKB) : Prop :=
  ∃ instruction data props,
    (.some ⟨ data, props ⟩ =
       Transpiler.transpile_op instruction entry.multiplicity
         #v[entry.pc_0, entry.pc_1, entry.pc_2]) ∧
    (entry = ProgramBusEntry.deserialise data) ∧ props
```

Two remarks on the formalisation are worth making. **First**, the axiomatic characterisation of the program counter is *enforced by the transpiler*, not by the CPU chip. The 4-byte alignment of pc_0 is exactly the property that the transpiler ensures during ELF processing (see the Pico Rust sources here and here), which is sound because Pico does not support the RISC-V compressed-instruction "C" extension. **Second**, the upper bound on the next PC (§1.2, Finding 1) must be enforced during transpilation to avoid finite-field overflow; this is documented at the equivalence layer in PicoFV/RV64IM/Equivalence/Equivalence.lean and threaded through as an explicit assumption.

2.3.2 The ALU bus

The ALU bus captures the result of every ALU-shaped opcode (R-type, immediate R-type, W-variants, and MUL/-DIV/REM families). Its entry, defined here, has a 13-element payload, consisting of one ALU opcode plus three operands of four u16 limbs each:

```
structure ALUBusEntry where
  multiplicity : F
  ALU_opcode   : F                    -- ALU opcode (range below)
  a_0 a_1 a_2 a_3 : F                 -- operands: a = b <op> c
  b_0 b_1 b_2 b_3 : F                 -- (each operand is a 4 × u16 limb word)
  c_0 c_1 c_2 c_3 : F
```

Externals and ranges. Not every ALU chip enforces operand u16 bounds internally. To capture this cleanly we introduce two predicates: validOpcode (the set of valid ALU opcodes) and operandsInternal (the subset of ALU opcodes whose chip range-checks its operand limbs itself). For opcodes outside operandsInternal, the calling air must assert the u16 ranges as part of the bus externals. All of this is defined here:

```
def ALUBus.validOpcode (ALU_opcode : FKB) : Prop :=
  ALU_opcode ∈ Finset.Icc 0 9   ∨   -- 64-bit R-type ALU
  ALU_opcode ∈ Finset.Icc 30 37 ∨   -- MUL / DIV / REM (64-bit)
  ALU_opcode ∈ Finset.Icc 40 44 ∨   -- W-variant R-type (RV64-only)
  ALU_opcode ∈ Finset.Icc 48 52     -- W-variant MUL / DIV / REM (RV64-only)

def ALUBus.operandsInternal (ALU_opcode : FKB) : Prop :=
  -- The chips for these opcodes prove u16 bounds on b/c themselves:
  --   - Bitwise (XOR/OR/AND) byte-bus AND/OR/XOR lookups carry u8 bounds.
  --   - Mul / MulW gadgets emit (u8, (limb - u8) · inv 256) for each limb.
  ALU_opcode ∈ Finset.Icc 2 4 ∨ ALU_opcode ∈ Finset.Icc 30 33 ∨ ALU_opcode = 48

def ALUBus.operandRange (b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3 : FKB) : Prop :=
  b_0.val < 65536 ∧ b_1.val < 65536 ∧ b_2.val < 65536 ∧ b_3.val < 65536 ∧
  c_0.val < 65536 ∧ c_1.val < 65536 ∧ c_2.val < 65536 ∧ c_3.val < 65536

def ALUBus.externals (entry : ALUBusEntry FKB) : Prop :=
  ¬(entry.multiplicity = 0) →
    ALUBus.validOpcode entry.ALU_opcode ∧
    (¬ ALUBus.operandsInternal entry.ALU_opcode →
       ALUBus.operandRange entry.b_0 entry.b_1 entry.b_2 entry.b_3 entry.c_0 entry.c_1 entry.c_2 entry.c_3)
```

Pure specification. The well-formedness predicate of every entry encodes the precise arithmetic relation the chip emitting it must satisfy. It defers to the pure functions defined in `PicoFV/RV64IM/RVSail/Execution.lean`. The specification is defined here:

```
def ALUBus.Spec (ALU_opcode a_0 a_1 a_2 a_3 b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3 : FKB) :=
  let a := U64w.toBV #v[a_0, a_1, a_2, a_3]
  let b := U64w.toBV #v[b_0, b_1, b_2, b_3]
  let c := U64w.toBV #v[c_0, c_1, c_2, c_3]
  let result_range  := a_0.val < 65536 ∧ a_1.val < 65536 ∧ a_2.val < 65536 ∧ a_3.val < 65536
  let operand_range := ALUBus.operandRange b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3
  result_range ∧
  match ALU_opcode with
  -- 64-bit R-type ALU (AddChip, SubChip, BitwiseChip, SLLChip, ShiftRightChip, LtChip).
  | 0 ⇒ a = RV64IM.execute_RTYPE_pure b c .ADD
  | 1 ⇒ a = RV64IM.execute_RTYPE_pure b c .SUB
  | 2 ⇒ operand_range ∧ a = RV64IM.execute_RTYPE_pure b c .XOR  -- chip enforces operand u16
  | 3 ⇒ operand_range ∧ a = RV64IM.execute_RTYPE_pure b c .OR
  | 4 ⇒ operand_range ∧ a = RV64IM.execute_RTYPE_pure b c .AND
  | 5 ⇒ a = RV64IM.execute_RTYPE_pure b c .SLL
  | 6 ⇒ a = RV64IM.execute_RTYPE_pure b c .SRL
  | 7 ⇒ a = RV64IM.execute_RTYPE_pure b c .SRA
  | 8 ⇒ a = RV64IM.execute_RTYPE_pure b c .SLT
  | 9 ⇒ a = RV64IM.execute_RTYPE_pure b c .SLTU
  -- MUL family (64-bit). operand_range in-spec; chip enforces internally.
  | 30 ⇒ operand_range ∧ a = RV64IM.execute_MUL_pure b c .MUL
  | 31 ⇒ operand_range ∧ a = RV64IM.execute_MUL_pure b c .MULH
  | 32 ⇒ operand_range ∧ a = RV64IM.execute_MUL_pure b c .MULHU
  | 33 ⇒ operand_range ∧ a = RV64IM.execute_MUL_pure b c .MULHSU
  -- DIV / REM (64-bit).
  | 34 ⇒ a = (RV64IM.execute_DIV_REM_pure b c .DRS).1
  | 35 ⇒ a = (RV64IM.execute_DIV_REM_pure b c .DRU).1
  | 36 ⇒ a = (RV64IM.execute_DIV_REM_pure b c .DRS).2
  | 37 ⇒ a = (RV64IM.execute_DIV_REM_pure b c .DRU).2
  -- W-variant R-type: operate on the low 32 bits of each operand, sign-extend the result.
  | 40 ⇒ a = RV64IM.execute_RTYPEW_pure (b.setWidth 32) (c.setWidth 32) .ADDW
  | 41 ⇒ a = RV64IM.execute_RTYPEW_pure (b.setWidth 32) (c.setWidth 32) .SUBW
  | 42 ⇒ a = RV64IM.execute_RTYPEW_pure (b.setWidth 32) (c.setWidth 32) .SLLW
  | 43 ⇒ a = RV64IM.execute_RTYPEW_pure (b.setWidth 32) (c.setWidth 32) .SRLW
  | 44 ⇒ a = RV64IM.execute_RTYPEW_pure (b.setWidth 32) (c.setWidth 32) .SRAW
  -- W-variant MUL / DIV / REM.
  | 48 ⇒ operand_range ∧ a = RV64IM.execute_MULW_pure (b.setWidth 32) (c.setWidth 32)
  | 49 ⇒ a = (RV64IM.execute_DIVW_REMW_pure (b.setWidth 32) (c.setWidth 32) .DRS).1
  | 50 ⇒ a = (RV64IM.execute_DIVW_REMW_pure (b.setWidth 32) (c.setWidth 32) .DRU).1
  | 51 ⇒ a = (RV64IM.execute_DIVW_REMW_pure (b.setWidth 32) (c.setWidth 32) .DRS).2
  | 52 ⇒ a = (RV64IM.execute_DIVW_REMW_pure (b.setWidth 32) (c.setWidth 32) .DRU).2
  | _  ⇒ True
```

The instance itself, defined here, plugs `ALUBus.externals` into `externals` and `ALUBus.Spec` into `wf_properties`. The design has two virtues directly relevant for formal verification:

- the ALU bus is a clean interface: any opcode that needs an ALU result simply reads an ALU entry, and the responsibility for the arithmetic is delegated to the ALU chip that wrote the entry; and
- the verification on each side becomes purely local: the ALU chip must prove `Spec` (assuming the calling chip provides the externals); the calling chip must prove the externals (and may then use `Spec`).

2.3.3 The `Byte` bus

The `Byte` bus aggregates the small range/bit-level lookup tables. Its entry has a 5-element payload, defined here:

```
structure ByteBusEntry where
  multiplicity : F
  byte_opcode  : F
  x_0 x_1 x_2 x_3 : F                  -- bus arguments
```

Since the `Byte` bus derives its meaning from statically-constructed lookup tables, we do not re-prove the table entries on the Lean side; we just formalise the specification that each `(byte_opcode, args)` tuple satisfies, and assume it on read. The RV64 `Byte`-bus opcode set inherits the RV32 opcodes `0..8` verbatim and adds one new variant, `9 = BitRange`, to support shift-amount checks. The pure specification is defined here:

```
def ByteBus.Spec (byte_opcode x_0 x_1 x_2 x_3 : FKB) :=
  match byte_opcode with
  | 0 ⇒ x_0 = x_2 &&& x_3 ∧ x_0.val < 256 ∧ x_1 = 0 ∧ x_2.val < 256 ∧ x_3.val < 256  -- AND
  | 1 ⇒ x_0 = x_2 ||| x_3 ∧ x_0.val < 256 ∧ x_1 = 0 ∧ x_2.val < 256 ∧ x_3.val < 256  -- OR
  | 2 ⇒ x_0 = x_2 ^^^ x_3 ∧ x_0.val < 256 ∧ x_1 = 0 ∧ x_2.val < 256 ∧ x_3.val < 256  -- XOR
  -- Logical shift left, wrapping at 256 (so `x_0` is the low byte of `x_2 << x_3`).
  | 3 ⇒ x_0 = (x_2 <<< x_3) % 256 ∧ x_0.val < 256 ∧ x_1 = 0 ∧ x_2.val < 256 ∧ x_3.val < 256
  -- Logical shift right with carry: `x_0` is the shifted byte, `x_1` is the carry.
  | 4 ⇒ x_0 = x_2 >>> x_3 ∧ x_1 = x_2 - (x_0 <<< x_3) ∧
        x_0.val < 256 ∧ x_1.val < 256 ∧ x_2.val < 256 ∧ x_3.val < 256
  -- Unsigned less-than (predicate).
  | 5 ⇒ (x_0 = if x_2 < x_3 then 1 else 0) ∧ x_1 = 0 ∧ x_2.val < 256 ∧ x_3.val < 256
  -- Most-significant bit of a byte.
  | 6 ⇒ (x_0 = if x_2 < 128 then 0 else 1) ∧ x_0.val < 256 ∧ x_1 = 0 ∧ x_2.val < 256 ∧ x_3.val < 256
  | 7 ⇒ x_0 = 0 ∧ x_1 = 0 ∧ x_2.val < 256 ∧ x_3.val < 256                  -- u8 range
  | 8 ⇒ x_0.val < 65536 ∧ x_1 = 0 ∧ x_2 = 0 ∧ x_3 = 0                     -- u16 range
  -- BitRange (RV64 addition): `x_0 < 2^x_2`, with `x_2 ≤ 16`.
  | 9 ⇒ x_1 = 0 ∧ x_2.val ≤ 16 ∧ x_3 = 0 ∧ x_0.val < 2 ^ x_2.val
  | _ ⇒ True
```

The corresponding instance is defined here.

2.3.4 The `State` bus

The `State` bus captures every read and write that an opcode performs against the architectural state, both general-purpose registers and main memory. Its entry, defined here, has a 9-element payload:

```
structure StateBusEntry where
  multiplicity : F
  chunk        : F                    -- timestamp pair: (chunk, clk) totally orders
  clk          : F                    -- accesses for the memory-consistency argument.
  ptr_0 ptr_1 ptr_2 : F               -- address (u16 limbs; register accesses use only ptr_0)
  x_0 x_1 x_2 x_3 : F                 -- value read or written (4 × u16)
```

Each value limb is u16-bounded; this is the `wf_properties` predicate of the instance, defined here:

```
instance StateBusEntryInstance
: BusEntry FKB (StateBusEntry FKB) :=
{
  multiplicity := fun entry ⇒ entry.1,
  data_length := 9,
  data := fun ⟨_, chunk, clk, ptr_0, ptr_1, ptr_2, x_0, x_1, x_2, x_3⟩ ⇒
            #v[chunk, clk, ptr_0, ptr_1, ptr_2, x_0, x_1, x_2, x_3]
  axioms := fun _ ⇒ True
  externals := fun _ ⇒ True
  wf_properties :=
    fun ⟨ _, _, _, _, _, _, x_0, x_1, x_2, x_3 ⟩ ⇒
      x_0.val < 65536 ∧ x_1.val < 65536 ∧ x_2.val < 65536 ∧ x_3.val < 65536
  ...
}
```

The way the `State` bus separates registers from memory is by address: the first 32 addresses are interpreted as registers and the rest as memory. This is the design point flagged by Finding 3 (cf. §1.2): the chips emitting memory accesses on the `State` bus must enforce that the address is ≥ 32 to avoid colliding with the register-address region.

2.3.5 The `Memory` bus

The `Memory` bus is the high-level summary bus for memory accesses, that is, it describes the load/store opcode and address as it would be emitted by the source instruction, before the access is decomposed into byte-level `State`-bus

operations. Its entry, defined here, has a 25-element payload:

```
structure MemoryBusEntry where
  multiplicity : F
  mem_opcode   : F                                  -- memory opcode
  a_0 a_1 a_2 a_3 : F                               -- read/written value (4 × u16)
  b_0 b_1 b_2 b_3 : F                               -- address parts: addr = b + c
  c_0 c_1 c_2 c_3 : F
  a_is_0 : F                                        -- destination register is x0?
  -- Memory opcode selectors:
  is_lb is_lbu is_lh is_lhu is_lw is_lwu is_ld : F -- `is_lwu`, `is_ld` are RV64-only.
  is_sb is_sh is_sw is_sd : F                       -- `is_sd` is RV64-only.
```

Pure specification. The pure specification, defined here, captures four things: the result u16-range, the address range ($[2^{16}, 2^{48})$, with the low 64 KiB reserved as Pico's stack-guard region), the alignment requirement per opcode (2-, 4-, or 8-byte), and, for the load opcodes, the relationship between the read value and its sign-extension / zero-extension to 64 bits. The load-extension equations are gated on `a_is_0` $= 0$ (i.e. $rd \neq$ `x0`) because for $rd =$ `x0` the chip does not bind the value limbs (zeroing is enforced CPU-side):

```
def MemoryBus.Spec
  (mem_opcode a_0 a_1 a_2 a_3 b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3 a_is_0 : FKB) : Prop :=
  let a := U64w.toBV #v[a_0, a_1, a_2, a_3]
  let b := U64w.toBV #v[b_0, b_1, b_2, b_3]
  let c := U64w.toBV #v[c_0, c_1, c_2, c_3]
  let addr := (b + c).toNat
  -- For load opcodes (10..14, 45, 46), the four value limbs are u16-bounded.
  ((mem_opcode.val ≤ 14 ∨ mem_opcode.val = 45 ∨ mem_opcode.val = 46) →
       a_0.val < 65536 ∧ a_1.val < 65536 ∧ a_2.val < 65536 ∧ a_3.val < 65536) ∧
  -- Address lives in `[2^16, 2^48)` (low 64 KiB reserved by MemoryReadWriteChip).
  65536 ≤ addr ∧ addr < Pico_address_space_size_64 ∧
  -- Alignment.
  (mem_opcode = 11 ∨ mem_opcode = 14 ∨ mem_opcode = 16 → 2 ∣ addr) ∧            -- LH/LHU/SH
  (mem_opcode = 12 ∨ mem_opcode = 17 ∨ mem_opcode = 45 → 4 ∣ addr) ∧            -- LW/SW/LWU
  (mem_opcode = 46 ∨ mem_opcode = 47 → 8 ∣ addr) ∧                              -- LD/SD
  -- Load-extension equations (only when rd ≠ x0).
  (mem_opcode = 10 → a_is_0 = 0 → a = BitVec.signExtend 64 (a.truncate 8))  ∧   -- LB
  (mem_opcode = 11 → a_is_0 = 0 → a = BitVec.signExtend 64 (a.truncate 16)) ∧   -- LH
  (mem_opcode = 12 → a_is_0 = 0 → a = BitVec.signExtend 64 (a.truncate 32)) ∧   -- LW
  (mem_opcode = 13 → a_is_0 = 0 → a = BitVec.zeroExtend 64 (a.truncate 8))  ∧   -- LBU
  (mem_opcode = 14 → a_is_0 = 0 → a = BitVec.zeroExtend 64 (a.truncate 16)) ∧   -- LHU
  (mem_opcode = 45 → a_is_0 = 0 → a = BitVec.zeroExtend 64 (a.truncate 32))     -- LWU
  -- LD (46) needs no extension; SB/SH/SW/SD (15-17, 47) don't constrain `a` on the bus.
```

Externals. The `externals` of the `MemoryBus` instance, defined here, fix the opcode to lie in the RV64 memory-opcode range $\{10, \ldots, 17\} \cup \{45, 46, 47\}$, u16-range-check both halves of the address pair, and enforce that the right one-hot selector is asserted for each opcode (and, for loads, that the value is zeroed when the destination is `x0`).

2.4 Bus operations

We now have a precise account of what each individual bus entry means in isolation. To formalise its effect on the Sail state we collect, for any one air row, the entries it produces or consumes on each of the five buses, and then translate that bundle into an `Operation` list. The first half is the `BusRow` record, defined here:

```
structure BusRow where
  Program : List (ProgramBusEntry FKB)
  ALU     : List (ALUBusEntry FKB)
  Byte    : List (ByteBusEntry FKB)
  State   : List (StateBusEntry FKB)
  Memory  : List (MemoryBusEntry FKB)
```

The second half is the translation `bus_operations`, defined here. It dispatches each bus entry into the appropriate `Operation`: a `Program` entry becomes a single `ReadPC`; each `State` entry becomes a register or memory read/write

depending on its address and multiplicity sign; each `Memory` entry becomes a width-appropriate memory read/write (or a `ReadMemAny` when the destination register is `x0`):

```
def bus_operations (row : BusRow) : List Operation :=
  -- Program bus: each entry contributes one ReadPC of its 3-limb encoded PC.
  let program : List Operation :=
    List.map (fun program_row ⇒
      .ReadPC (BitVec.ofNatLT (w := 64)
        (program_row.pc_0.val + 65536 * program_row.pc_1.val
          + 65536 * 65536 * program_row.pc_2.val)
        (by omega))
    ) row.Program
  -- State bus: register access if addr < 32, otherwise a memory access.
  let state : List Operation :=
    List.filterMap (fun state_row ⇒
      let value := U64w.toBV #v[state_row.x_0, state_row.x_1, state_row.x_2, state_row.x_3]
      let addr  := state_row.ptr_0.val + 65536 * state_row.ptr_1.val
                 + 65536 * 65536 * state_row.ptr_2.val
      if _ : addr < 32 then
        let register : regidx := .Regidx (BitVec.ofNatLT (w := 5) addr (by omega))
        if (state_row.multiplicity = -1) then .some (.ReadReg register value)
        else if (state_row.multiplicity = 1) then .some (.WriteReg register value)
        else .none
      else
        -- Memory entries on the State bus are always full 8-byte Doubles
        -- (the State bus is the byte-level commitment; the Memory bus carries
        -- the high-level shape).
        if (state_row.multiplicity = -1) then .some (.ReadMem addr .Double value)
        else if (state_row.multiplicity = 1) then .some (.WriteMem addr .Double value)
        else .none
    ) row.State
  -- Memory bus: pick the appropriate width from the selectors, dispatch on
  -- store vs. load, and use ReadMemAny when the load destination is x0.
  let memory : List Operation :=
    List.filterMap (fun (memory_row : MemoryBusEntry FKB) ⇒
      let width : Option MemoryWidth :=
        if (memory_row.is_ld = 1 ∨ memory_row.is_sd = 1) then .some .Double
        else if (memory_row.is_lw = 1 ∨ memory_row.is_lwu = 1 ∨ memory_row.is_sw = 1) then .some .Word
        else if (memory_row.is_lh = 1 ∨ memory_row.is_lhu = 1 ∨ memory_row.is_sh = 1) then .some .Half
        else if (memory_row.is_lb = 1 ∨ memory_row.is_lbu = 1 ∨ memory_row.is_sb = 1) then .some .Byte
        else .none
      match width with
      | .none ⇒ .none
      | .some width ⇒
        let address := (U64w.toBV #v[memory_row.b_0, memory_row.b_1, memory_row.b_2, memory_row.b_3]
                      + U64w.toBV #v[memory_row.c_0, memory_row.c_1, memory_row.c_2, memory_row.c_3]).toNat
        let is_store : Bool :=
          memory_row.is_sb == 1 ∨ memory_row.is_sh == 1
            ∨ memory_row.is_sw == 1 ∨ memory_row.is_sd == 1
        match is_store with
        | true  ⇒ match width with
                  | .Byte   ⇒ .some (.WriteMem address .Byte   (BitVec.ofNat 8  (memory_row.a_0).val))
                  | .Half   ⇒ .some (.WriteMem address .Half   (BitVec.ofNat 16 (memory_row.a_0).val))
                  | .Word   ⇒ .some (.WriteMem address .Word
                                      ((BitVec.ofNat 16 (memory_row.a_1).val)
                                      ++ (BitVec.ofNat 16 (memory_row.a_0).val)))
                  | .Double ⇒ .some (.WriteMem address .Double
                                      (U64w.toBV #v[memory_row.a_0, memory_row.a_1,
                                                    memory_row.a_2, memory_row.a_3]))
        | false ⇒ if memory_row.a_is_0 = 1 then
                    -- rd = x0: spec records that the bytes exist, not their value.
                    .some (.ReadMemAny address width)
                  else match width with
                    | .Byte   ⇒ .some (.ReadMem address .Byte   (BitVec.ofNat 8  (memory_row.a_0).val))
                    | .Half   ⇒ .some (.ReadMem address .Half   (BitVec.ofNat 16 (memory_row.a_0).val))
```

```
                    | .Word   ⇒ .some (.ReadMem address .Word
                                       ((BitVec.ofNat 16 (memory_row.a_1).val)
                                       ++ (BitVec.ofNat 16 (memory_row.a_0).val)))
                    | .Double ⇒ .some (.ReadMem address .Double
                                       (U64w.toBV #v[memory_row.a_0, memory_row.a_1,
                                                     memory_row.a_2, memory_row.a_3]))
    ) row.Memory
  program ++ state ++ memory
```

The output of bus_operations is a fully concrete operation list, ready to be fed through operation_list_effect of §2.1 and compared against the per-opcode spec_* list.

2.5 Opcode verification workflow

With the above infrastructure in place, verifying one opcode follows the same five-step recipe in every case:

1. implement the transpilation of the opcode in Transpiler.transpile_op;
2. formalise the chip that implements the opcode (with its column and row layout) under PicoFV/RV64IM/Airs/;
3. extract the chip's constraints and bus contents from the Pico Rust source, under PicoFV/RV64IM/Extraction/;
4. transform the extracted constraints into a human-readable form under PicoFV/RV64IM/Constraints/; and
5. prove that, on any chip row, the operation-list effect of the opcode's spec_* agrees with the operation-list effect of the chip's bus_operations: the per-opcode equivalence theorem in Equivalence.lean.

The 62 RV64IM correctness theorems all live in Equivalence.lean. As an illustration, here is the ADD theorem, defined here, annotated for readability:

```
theorem equivalence_ADD
  [Field ExtF]
  -- Let `row` be a non-padding, in-range row of the CPU chip `air`,
  (air : Valid_CpuChip FKB ExtF)
  (row : ℕ)
  (h_row : row ≤ air.last_row)
  (h_is_real : air.cols.is_real row 0 = 1)
  -- and assume the bus-row axioms / externals / well-formedness on every row,
  (h_all_axioms     : ∀ row ≤ air.last_row, bus_row_axioms (bus_row air row))
  (h_all_ext        : ∀ row ≤ air.last_row, bus_row_ext_asm (bus_row air row))
  (h_all_wf         : ∀ row ≤ air.last_row, bus_row_wf_asm (bus_row air row))
  (h_all_wf_asrt    : ∀ row ≤ air.last_row, bus_row_wf_asrt (bus_row air row))
  -- the chip's row constraints,
  (h_all_constraints : ∀ row ≤ air.last_row, CpuChip.Extraction.allHold air row)
  -- and that the row is an `ADD` (opcode 0, no immediate on the c-operand).
  (h_opcode : air.cols.instruction.opcode row 0 = 0 ∧ air.cols.opcode_selector.imm_c row 0 = 0)
  -- Further, assume that the hypotheses of the CPU bus row operations hold,
  (h_bus_hypotheses :
    (operation_list_effect (CPU_bus_operations air row) state).hypotheses)
  -- and that the Sail-state assumptions of \S\ref{sec:caveats} hold.
  (_risc_v_assumptions : RISC_V_assumptions state mstatus pmaRegion misa mseccfg)
:
  -- Then executing the `ADD` instruction has the same post-state as applying
  -- the chip-row's bus operations.
  let rd  : regidx := .Regidx (air.cols.instruction.op_a_0 row 0).val
  let rs1 : regidx := .Regidx (air.cols.instruction.op_b_0 row 0).val
  let rs2 : regidx := .Regidx (air.cols.instruction.op_c_0 row 0).val
  execute_instruction (instruction.RTYPE (rs2, rs1, rd, rop.ADD)) state =
    (operation_list_effect (CPU_bus_operations air row) state).state
```

The corresponding RV32IM theorem lives under the parallel RV32IM/Equivalence directory and is structurally identical, with BitVec 64 replaced by BitVec 32 and the W-variant theorems absent. The full list of the 45 RV32IM theorems and the 62 RV64IM theorems is collected in the respective Equivalence.lean files; together with the rest of this section's infrastructure, they constitute the deliverable described in §1, item 3.

3 From Pico chips to readable Lean

In this section we look inside the three directories that, together, take a chip from the Pico Rust sources into the form that the per-opcode equivalence proofs of §2.5 can actually consume: `PicoFV/RV64IM/Extraction/`, `PicoFV/RV64IM/Airs/`, and `PicoFV/RV64IM/Constraints/`. We illustrate each layer with the `AddChip`, which is small enough to fit in this section in full. The same pattern scales up unchanged to the larger chips (the CPU chip, for example, follows it across 218 constraints).

To anchor the discussion: each Pico chip is a small Plonky3 air defined in the Pico Rust sources, with three things attached to it: a column layout (a Rust struct), a set of algebraic constraints (typically built up by an `eval`-style function), and a list of bus interactions (LogUp lookups into the five buses of §2.2). The same triad ends up split across `Extraction/`, `Airs/`, and `Constraints/`, with each folder taking responsibility for a different part of the work.

3.1 Extraction: the raw constraint dump

The `Extraction/` directory is the only one that is not hand-written: every file in it is the output of an automated extractor we built that walks the Pico Rust source, runs the air's `eval` function symbolically, and records the resulting Plonky3 constraint expressions and bus interactions in Lean syntax. The mechanism of the extractor itself is out of scope for this report; we focus here on what it produces and on how the rest of the infrastructure interfaces with it.

Each extracted chip file is a single Lean module under `PicoFV/RV64IM/Extraction/` that has a fixed, repetitive shape:

- one header comment line identifying the extraction timestamp and the Pico Rust commit hash from which the chip was extracted;
- one numbered `constraint_i` definition per algebraic constraint emitted by the chip's `eval` function (every constraint is tagged with the `@[extracted_constraint_definition]` simp attribute);
- one `constrain_interactions` definition collecting every bus interaction the chip emits across all rows, with each entry as a `(multiplicity, payload)` tuple;
- one `constraint_list` aggregating all `constraint_i`s into a `List Prop`; and
- one `allHold` predicate combining `constrain_interactions` with the conjunction of every constraint in `constraint_list`.

To get a feel for the shape of the output, the extracted `AddChip` opens as follows:

```
--Extracted on 2026-5-11 at 15:36:49.420173697 using git commit "e1280624bd"
import Mathlib.Data.List.Defs
import LeanZKCircuit_Plonky3.Plonky3.Circuit
import PicoFV.RV64IM.Extraction.Attributes
open Plonky3
set_option linter.all false

namespace AddChip.Extraction
--Num constraints: 6
--Constraints---
@[extracted_constraint_definition]
def constraint_0 {C : Type → Type → Type} {F ExtF : Type}
                 [Field F] [Field ExtF] [Circuit F ExtF C]
                 (c : C F ExtF) (row : ℕ) :=
  ((Circuit.main c (column := 12) (row := row) (rotation := 0))
    * ((Circuit.main c (column := 12) (row := row) (rotation := 0)) - 1)) = 0
```

and the third constraint (limb-1 carry-boolean check) already shows the characteristic shape:

```
@[extracted_constraint_definition]
def constraint_2 {C : Type → Type → Type} {F ExtF : Type}
                 [Field F] [Field ExtF] [Circuit F ExtF C]
                 (c : C F ExtF) (row : ℕ) :=
  ((Circuit.main c (column := 12) (row := row) (rotation := 0))
    * ((((((Circuit.main c (column := 5)  (row := row) (rotation := 0))
           + (Circuit.main c (column := 9) (row := row) (rotation := 0)))
       - (Circuit.main c (column := 1) (row := row) (rotation := 0)))
```

```
      + (((((Circuit.main c (column := 4) (row := row) (rotation := 0))
            + (Circuit.main c (column := 8) (row := row) (rotation := 0)))
          - (Circuit.main c (column := 0) (row := row) (rotation := 0)))
          + 0) * 2130673921)) * 2130673921)
      * ((((((Circuit.main c (column := 5) (row := row) (rotation := 0))
              + (Circuit.main c (column := 9) (row := row) (rotation := 0)))
            - (Circuit.main c (column := 1) (row := row) (rotation := 0)))
          + (((((Circuit.main c (column := 4) (row := row) (rotation := 0))
                + (Circuit.main c (column := 8) (row := row) (rotation := 0)))
              - (Circuit.main c (column := 0) (row := row) (rotation := 0)))
              + 0) * 2130673921)) * 2130673921) - 1))) = 0
```

Three properties of the extracted form are worth highlighting:

1. **Columns are referenced by integer index** via `Circuit.main c (column := n) (row := row) (rotation := r)`. The integer `n` is the position of the column in the chip's main grid, counting from zero in the order the Pico Rust struct emits them. There are no names: column 0 of `AddChip` is *some* field of the underlying Rust struct, but the extracted file makes no commitment to which. Untangling that mapping is the job of §3.2.
2. **Magic numbers appear inline.** The constant `2130673921` in the snippet is the multiplicative inverse of 2^{16} in the KoalaBear field, appearing because the Pico `AddGadget` reconstructs the limb-i carry as `carry_i` $=$ (limb diff $+$ carry_i-1) $\cdot\, 2^{-16}$. Such constants are inlined directly, without any commentary.
3. **Nesting depth scales with constraint complexity.** The carry chain in `constraint_2` above already nests two limbs deep; by `constraint_4` (the limb-3 check) the same expression appears with four levels of carry nesting. The deepest extracted constraint in the CPU chip nests several dozen levels.

The bus interactions are emitted next, all in one `constrain_interactions` definition that simply lists every bus entry the chip's `eval` emits per row:

```
@[extracted_constraint_definition]
def constrain_interactions {C : Type → Type → Type} {F ExtF : Type}
                           [Field F] [Field ExtF] [Circuit F ExtF C]
                           (c : C F ExtF) :=
  Circuit.bus c = (List.range (Circuit.last_row c + 1)).flatMap (λ row ⇒
    [ (-((Circuit.main c (column := 12) (row := row) (rotation := 0))),
       [4, 8, (Circuit.main c (column := 0) (row := row) (rotation := 0)), 0, 0, 0]),
      (-((Circuit.main c (column := 12) (row := row) (rotation := 0))),
       [4, 8, (Circuit.main c (column := 1) (row := row) (rotation := 0)), 0, 0, 0]),
       ... ])
```

Each entry is a `(multiplicity, payload)` pair. The leading literal of the payload identifies the bus (here `4` = `Byte` bus, `3` = `ALU` bus, etc.), and the rest of the payload is the bus-specific data, exactly as the Rust source emits them.

Finally, the file closes by aggregating constraints and bus interactions into an "everything holds on this row" predicate:

```
@[constraint_and_interaction_simplification]
def constraint_list ... : List Prop :=
  [constraint_0 c row, constraint_1 c row, ...]

@[constraint_and_interaction_simplification]
def allHold ... : Prop :=
  constrain_interactions c ∧ List.Forall (·) (constraint_list c row)
```

The `allHold` predicate is the single hypothesis that every downstream equivalence proof takes as its "the chip's constraints fire on this row". The two custom simp attributes `@[extracted_constraint_definition]` (on the raw constraints) and `@[constraint_and_interaction_simplification]` (on the aggregator and on its named-accessor counterparts in `Constraints/`) are the hinges that the simplification step in §3.3 relies on, defined here in `Extraction/Attributes.lean`.

The extracted file is, by design, both opaque and authoritative. It is opaque because columns have no names and constraints carry no semantic commentary; it is authoritative because, by construction, it is a verbatim record of what the chip's `eval` function evaluates symbolically. Every later layer of the development must reconcile itself with this raw form, never the other way around.

3.2 Airs: re-typing chip columns with their Rust names

The `Airs/` directory is where the extracted columns get their names back. For each chip we hand-write a file under `PicoFV/RV64IM/Airs/` that reproduces the chip's Rust column-layout struct as a Lean "sub-air". The two custom commands that do this work, `#define_subair` and `#define_air`, come from the underlying LeanZKCircuit-Plonky3 package and are configured against our `pico_encapsulation` simp set so that, after definition, every column accessor unfolds back to a `Circuit.main c (column := n) ...` call, recovering the raw form of §3.1 when needed.

For the `AddChip`, the column layout in Pico's Rust source `vm/src/chips/chips/alu/add/columns.rs` is:

```
pub struct AddCols<F> {
    pub values: [AddValueCols<F>; ADD_DATAPAR],   // ADD_DATAPAR = 1
}

pub struct AddValueCols<F> {
    pub add_operation: AddGadget<F>,              // 4 cols (Word<F>)
    pub operand_1: Word<F>,                       // 4 cols
    pub operand_2: Word<F>,                       // 4 cols
    pub is_add: F,                                // 1 col
}                                                 // total: 13 cols
```

The corresponding Lean file at `Airs/AddChip.lean` mirrors the struct structurally:

```
#define_subair "AddValueCols" using "pico_encapsulation" where
  SubAir["add_operation" : "AddGadget" width := 4]
  Column["operand_1_0"]
  Column["operand_1_1"]
  Column["operand_1_2"]
  Column["operand_1_3"]
  Column["operand_2_0"]
  Column["operand_2_1"]
  Column["operand_2_2"]
  Column["operand_2_3"]
  Column["is_add"]

#define_subair "AddCols" using "pico_encapsulation" where
  SubAir["values" : "AddValueCols" width := 13]

#define_air "AddChip" using "pico_encapsulation" where
  MainSubAir["cols" : "AddCols" width := 13]
```

Two design decisions drive the look of these files. **First, the field path is exactly the Rust field path.** Where Pico's Rust code reads `cols.values.add_operation.value_0` ("the chip's `cols` grid, projected to its `values` sub-struct, projected to the embedded `add_operation` (`AddGadget`), and finally to its first limb `value_0`"), our Lean code reads `air.cols.values.add_operation.value_0 row 0`. The Lean projection order matches the Rust one field-for-field. Every chip file in `Airs/` carries, in its Lean docstring, a verbatim transcript of the Rust struct it is mirroring (cf. the `AddCols` block above), so that the field-path correspondence can be checked at a glance against the source. **Second, structure follows the chip's compositional decomposition.** `AddCols` contains an `AddValueCols`; `AddValueCols` embeds an `AddGadget`. The Lean side mirrors this by introducing one `#define_subair` per Rust struct and stacking them: at the bottom `AddGadget` (defined in its own file at `Airs/AddGadget.lean`), above it `AddValueCols`, above that `AddCols`, and finally `#define_air "AddChip"` for the top-level air. This means every gadget shared between several chips (the `AddGadget` above; the `IsZeroGadget`, `U16ToU8GadgetSafe`, `MulGadget`, etc.) appears once and is re-used: the consumer chip just embeds it via `SubAir["<field>" : "<GadgetName>" width := <gadget cols>]`, with the field name on the chip and the type name on the gadget, exactly mirroring the named-field embedding of Rust.

Sub-airs that occur in more than one chip, such as `MemoryAccessCols` / `MemoryReadCols` / `MemoryWriteCols` / `MemoryReadWriteCols` from `vm/src/chips/chips/riscv_memory/read_write/columns.rs`, used by both the `CpuChip` and the `MemoryReadWriteChip`, all live in `Airs/SubAirs.lean`.

The `#define_air` command does two further pieces of bookkeeping that the rest of the development relies on. First, it generates a `Valid_AddChip` wrapper type that pairs the raw air with a proof of *validity*, namely that the chip respects the standard Plonky3 well-formedness invariants (e.g. the column-rotation periodicity used by `pc_0_rotation` and its

analogues in `Constraints/CpuChipInterpretations.lean`). Second, it generates an `air.cols.<field>` accessor for every field path in the underlying sub-air tree. The end result is that, for the `AddChip`:

- the Rust expression `cols.values.is_add` corresponds to the Lean accessor `air.cols.values.is_add row 0`;
- this accessor unfolds, via the `@[pico_encapsulation]` simp set, to `Circuit.main air (column := 12) (row := row) (rotation := 0)` , the very expression used by the extracted constraints in §3.1;
- and conversely, any extracted constraint can be re-expressed in terms of named accessors by folding the same simp set in reverse.

This last bidirectional property is the bridge that makes the simplification step of §3.3 go through as a one-line `simp`.

3.3 Constraints: from raw extraction to human-usable form

The `Constraints/` directory is the layer that pulls the extraction together with the airs and produces a chip-level statement that can actually be used in a proof. For each chip there is a single file under `PicoFV/RV64IM/Constraints/` that does five things, in the same order in every file. We walk through each step using the `AddChip` again, with the corresponding source at `Constraints/AddChip.lean`.

Step 1: named-accessor constraints. For each extracted `constraint_i`, the `Constraints/` file defines a parallel `ConstraintsAndBuses.constraint_i` that asserts exactly the same algebraic identity, but written in terms of the named accessors of §3.2. For the `AddChip`'s limb-0 carry check this looks like:

```
@[constraint_and_interaction_simplification]
def constraint_1 (air : Valid_AddChip F ExtF) (row : ℕ) : Prop :=
  ((air.cols.values.is_add row 0)
    * ((((((air.cols.values.operand_1_0 row 0)
            + (air.cols.values.operand_2_0 row 0))
        - (air.cols.values.add_operation.value_0 row 0))
      + 0) * 2130673921)
      * ((((((air.cols.values.operand_1_0 row 0)
              + (air.cols.values.operand_2_0 row 0))
          - (air.cols.values.add_operation.value_0 row 0))
        + 0) * 2130673921) - 1))) = 0
```

The mapping from the extracted form is purely mechanical: column `0` becomes `add_operation.value_0`, column `4` becomes `operand_1_0`, column `8` becomes `operand_2_0`, column `12` becomes `is_add`, and so on. The structure of the expression is preserved verbatim: carries, parentheses, inverse-of-2^{16} constants and all. *No simplification of the arithmetic happens at this step*: the named-accessor form is still long and bracket-heavy, but every projection now reads like the Rust field it corresponds to.

The fact that the two forms denote the same proposition is then captured by a one-line "equivalence" lemma per constraint:

```
@[simp]
lemma constraint_1_equiv (air : Valid_AddChip F ExtF) (row : ℕ)
  : AddChip.Extraction.constraint_1 air row ↔ constraint_1 air row
:= by simp [pico_encapsulation,
            extracted_constraint_definition,
            constraint_and_interaction_simplification]
```

The proof is uniformly the same single `simp` call: it folds the named accessors back into their underlying `Circuit.main`-shaped raw form via the `pico_encapsulation` simp set generated by `#define_air` (§3.2), and unfolds both sides through the `@[extracted_constraint_definition]` and `@[constraint_and_interaction_simplification]` simp attributes. Crucially, the simp set is constructed so that, after rewriting, the two sides become syntactically identical and `rfl`-closable, so the lemma is closed without any human-side algebra.

Step 2: named-accessor bus interactions. A parallel `raw_bus_row` definition unfolds the chip's extracted `constrain_interactions` into a hand-written list of `(multiplicity, payload)` tuples expressed via named accessors, defined for the `AddChip` here:

```
@[constraint_and_interaction_simplification]
```

```
def raw_bus_row (air : Valid_AddChip FKB ExtF) (row : ℕ) : List (FKB × List FKB) :=
  [ (-(air.cols.values.is_add row 0),
       [4, 8, (air.cols.values.add_operation.value_0 row 0), 0, 0, 0]),
    (-(air.cols.values.is_add row 0),
       [4, 8, (air.cols.values.add_operation.value_1 row 0), 0, 0, 0]),
    ...
    ((air.cols.values.is_add row 0),
       [3, 0, ...]) ]
```

The chip's raw bus row is then re-cast into the typed `BusRow` of §2.4 via a small `deserialise_bus_row` pass, and a per-chip `Interactions.bus_row` definition gives the typed result back.

Step 3: gadget abstraction. Constraints that come from an embedded gadget (the four limb-carry checks above all come from `AddGadget::eval`) are split off from the chip's own constraints and folded under the gadget's `AbsConstraints` predicate, defined once in the gadget's own `Constraints/` file. For the `AddChip`, the bridge is the lemma `constraint_subset_iff_gadget_00`:

```
@[simp]
lemma constraint_subset_iff_gadget_00 (air : Valid_AddChip F ExtF) (row : ℕ)
  : (constraint_0 air row ∧ constraint_1 air row ∧ constraint_2 air row
       ∧ constraint_3 air row ∧ constraint_4 air row)
    ↔
    AddGadget.AbsConstraints
      (air.cols.values.is_add row 0)
      (air.cols.values.operand_1_0 row 0) (air.cols.values.operand_1_1 row 0)
      (air.cols.values.operand_1_2 row 0) (air.cols.values.operand_1_3 row 0)
      (air.cols.values.operand_2_0 row 0) (air.cols.values.operand_2_1 row 0)
      (air.cols.values.operand_2_2 row 0) (air.cols.values.operand_2_3 row 0)
      (air.cols.values.add_operation.value_0 row 0)
      (air.cols.values.add_operation.value_1 row 0)
      (air.cols.values.add_operation.value_2 row 0)
      (air.cols.values.add_operation.value_3 row 0)
:= by simp [AddGadget.AbsConstraints, constraint_and_interaction_simplification]
```

The shared `AddGadget.AbsConstraints` predicate captures, parametrically in the column values, the conjunction of algebraic facts that the gadget asserts; the same predicate is then re-used by every chip embedding the gadget, with each chip merely renaming `is_real` ↦ `is_add`, `b_i` ↦ `operand_1_i`, `c_i` ↦ `operand_2_i`, `value_i` ↦ `add_operation.value_i`. The same pattern factors out `AddGadget.AbsBuses` for the bus contribution. The upshot is that any chip embedding the gadget gets its limb-carry constraints *and* its byte-bus range-check entries for free, via a single rewrite into the gadget's `AbsConstraints ∧ AbsBuses`; the only chip-side work is to discharge whatever extra constraints and bus entries the chip itself adds (here just the duplicate boolean check on `is_add` and the chip's outgoing `ALU` bus entry).

Step 4: aggregation. A single per-chip `allHold_simplified` predicate aggregates the named-accessor constraint list with the chip's typed bus row, and a one-line lemma proves that the extracted `allHold` of §3.1 is iff this simplified form:

```
@[constraint_and_interaction_simplification]
def allHold_simplified (air : Valid_AddChip FKB ExtF) (row : ℕ) : Prop :=
  constrain_interactions air ∧ List.Forall (·) (row_constraint_list air row)

@[simp]
lemma allHold_iff_allHold_simplified
  (air : Valid_AddChip FKB ExtF) (row : ℕ)
: AddChip.Extraction.allHold air row ↔ allHold_simplified air row
:= by simp [pico_encapsulation,
            extracted_constraint_definition,
            constraint_and_interaction_simplification]
```

After this lemma, the consumer (typically the per-opcode equivalence proof in `Equivalence.lean`) never has to mention the extracted form again: it takes `AddChip.Extraction.allHold air row` as a hypothesis (this is the shape

§2.5 arranges for), and rewrites once via `allHold_iff_allHold_simplified` to obtain a named-accessor-shaped, gadget-abstracted conjunction.

Step 5: interpretations. Finally, the `<Chip>.Interpretations` namespace contains the semantic "packagings" the chip exposes for use by other chips that consume it: typically a per-column rotation-invariance lemma, an `allHold_abs` that bundles all the constraints into a single gadget-abstracted statement, and a `bus_row_abs` that does the same for the bus row. For the `AddChip`, the typical interpretation lemma reads:

```
lemma allHold_abs
  [Field ExtF]
  (air : Valid_AddChip FKB ExtF)
  (row : ℕ)
  (h_constraints : AddChip.Extraction.allHold air row)
:
  AddGadget.AbsConstraints
    (air.cols.values.is_add row 0)
    (air.cols.values.operand_1_0 row 0) (air.cols.values.operand_1_1 row 0)
    (air.cols.values.operand_1_2 row 0) (air.cols.values.operand_1_3 row 0)
    (air.cols.values.operand_2_0 row 0) (air.cols.values.operand_2_1 row 0)
    (air.cols.values.operand_2_2 row 0) (air.cols.values.operand_2_3 row 0)
    (air.cols.values.add_operation.value_0 row 0)
    (air.cols.values.add_operation.value_1 row 0)
    (air.cols.values.add_operation.value_2 row 0)
    (air.cols.values.add_operation.value_3 row 0)
  ∧ constraint_5 air row
```

On the CPU chip side, this same pattern blooms into the much larger `Constraints/CpuChipInterpretations.lean` file, which exposes both per-column rotation lemmas and a long catalogue of "the chip implies this semantic fact at this row" interpretations (typed `BusRow`, `program_bus_entry`, the `U64w`-wrapped `pc` and `next_pc`, and so on) that are then directly consumed by the per-opcode equivalence theorems of §2.5.

Summary. The three folders form a one-way pipeline: `Extraction/` produces a raw, opaque transcript of the chip in column-index form; `Airs/` provides the Rust-field-faithful names and the validity wrapper around it; `Constraints/` reconciles the two via per-constraint and per-bus-row equivalence lemmas, factors out shared gadget behaviour into reusable `AbsConstraints` / `AbsBuses` predicates, and finally exposes a small set of semantic interpretations that downstream proofs can consume directly. The mechanical-to-semantic gap is bridged in a handful of `simp`-only lemmas per chip; the human work (choosing the right names, the right sub-air decomposition, and the right semantic interpretations to expose) is all that remains.

4 Disclaimer

This Report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the tasks outlined in the Executive Summary. The content of this Report is intended to provide an analysis of the formal verification process undertaken, and the results obtained, based on the information and data available to Nethermind up to the time of completion. The formal verification process conducted by the team of engineers was based on the specific parameters, methodologies, and data available at the time of the analysis. Any changes in the underlying parameters, methodologies, or data may lead to different results.

Nethermind has provided the review and this Report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to any other areas outside the scope of work agreed with you in our Agreement. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the Report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.